

UNIVERSITY OF GHANA
DEPARTMENT OF COMPUTER SCIENCE

DCIT 208 – Software Engineering

Individual Assignment 4: Architecture Decision Record and Failure Walkthrough

A. Student and project identification

Full name	AIDOO ROBERT KWABENA
Student ID	22337992
Team name	TechCare
Project title	Pharmacy Web Application
Team repository URL	https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-23techcare
Individual assignment repository URL	https://github.com/Dept-of-Comp-Sci-University-of-Ghana/ia4-architecture-decision-record-and-failure-walkthrough-AJ2025LEG

B. Decision title and status

ADR-001: Use managed cloud object storage instead of local server file storage for prescription receipt uploads.

Status: Proposed

C. Project context

The client problem: prescription-only medications cannot be dispensed without a valid prescription. My IA3 feature (Customer uploads a prescription receipt for verification) lets a customer attach an image/PDF of their prescription to an order, which a pharmacist then reviews before the order is approved.

Selected features: prescription receipt upload and storage, feeding the pharmacist verification queue.

Current requirement/NFR: from IA3, the linked NFR is Security & Privacy — only the uploading customer and authorized pharmacy staff should be able to access a given prescription file, and access should be logged. There is also an implicit reliability requirement: once a customer uploads a prescription, it must not be lost before the pharmacist reviews it.

Why this decision matters now: the team is finalizing the backend for the upload feature and must decide, before writing the storage code, whether uploaded files are saved directly on the application server's disk or sent to a managed storage provider. This choice affects the database schema (what a 'file_path' points to), the review workflow, and how much custom access-control code the team has to write and maintain.

D. Assumptions and constraints

- Users: customers and pharmacists interact through a browser on desktop or mobile; customers may have inconsistent internet quality, so uploads should tolerate slower connections.
- Devices: prescription images are commonly phone camera photos (JPEG) or scanned PDFs.
- Data: files contain personal health information (prescription contents), so they are sensitive by default.
- Deployment: the team is deploying the project on a free-tier hosting platform for coursework demonstration, which typically has an ephemeral/non-persistent filesystem across restarts and redeploys.
- Cost: the team has no budget for paid infrastructure; any solution must work within a free tier.
- Team skills: the team is comfortable with Flask and SQLite but has limited prior experience with third-party storage SDKs.
- Deadlines: the project is on a fixed academic timeline, so the chosen approach must be implementable within the remaining sprint time.

- Privacy: only the customer who uploaded a file and authorized pharmacy staff should be able to view it.
- Expected scale: low volume for the course project (a small number of test customers and orders), not production pharmacy-chain scale.

E. Alternatives

Alternative A: Local file storage on the application server (rejected)

Uploaded files are saved directly to a folder on the same server that runs the Flask app, and the database stores a relative file path. A Flask route checks the requesting user's identity before serving the file.

Alternative B: Managed cloud object storage, e.g., Cloudinary free tier (chosen)

Uploaded files are sent to a third-party storage service through its SDK/API. The database stores the returned file URL or public ID instead of a local path. The application generates or retrieves a private/signed URL when an authorized user needs to view the file.

F. Comparison matrix

Criterion	Weight	A: Local server file storage	B: Managed cloud object storage (chosen)	Evidence / reasoning
Client/data fit	High	Files live on the same disk as the app; simple to code first	Files survive deploys and server restarts; matches a client who expects uploaded prescriptions to never disappear	Our free-tier host (Render/Railway-style) uses an ephemeral filesystem — a redeploy or restart wipes local uploads. A pharmacy client cannot accept lost prescription evidence.
Security/privacy	High	Access control must be hand-built entirely in our Flask routes; easy to accidentally leave a folder publicly servable	Provider gives private buckets/signed, time-limited URLs; access control still enforced in our route before generating a link	Prescription images are health-related data (per our IA3 NFR). Signed URLs plus our own auth check give two layers instead of one.
Maintainability	High	No extra dependency, but we must write and maintain our own storage-cleanup and backup logic	One SDK dependency (e.g., Cloudinary) to maintain; storage/backup handled by the provider	Team is 7 members with limited DevOps time; offloading storage housekeeping reduces code we must maintain ourselves.
Team skill	Medium	Matches what the team already knows (plain Flask + filesystem)	Requires learning a small SDK and free-tier account setup	Slight learning curve, but the Cloudinary/Flask integration is well documented and low-risk for a team already using Flask.
Deployment/cost	Medium	No extra cost, but breaks on redeploy on our chosen free hosting tier	Free tier is sufficient for project-scale usage (low volume of prescription images during the course project)	Free tier limits (storage/bandwidth) comfortably cover a student project demo and pilot volume.
Performance/scale	As relevant	Fine at small scale; no CDN, so images load	Provider CDN can serve images faster and does	Not a hard requirement now, but avoids our app

Criterion	Weight	A: Local server file storage	B: Managed cloud object storage (chosen)	Evidence / reasoning
		directly from our single server	not add load to our application server	server becoming a bottleneck if verification volume grows.

G. Decision and rationale

The team will use managed cloud object storage (Cloudinary free tier) for prescription receipt uploads instead of saving files directly on the application server's disk.

Rationale: the deciding factor is reliability under our actual deployment constraint. Because the team's free-tier host does not guarantee persistent disk storage across restarts or redeployments, local file storage risks silently losing a customer's uploaded prescription — which would break the verification workflow and could look, to the client, like data loss of sensitive medical evidence. Managed storage removes this risk without requiring the team to build or pay for our own persistent storage. It also gives built-in access control primitives (signed, time-limited URLs) that reduce how much custom security code the team has to write correctly by hand, which matters given the team's limited DevOps experience and fixed timeline.

H. Consequences and trade-offs

- Benefit: uploaded prescriptions persist reliably regardless of app server restarts or redeployments.
- Benefit: reduces custom access-control code the team must write and test, lowering the chance of an access-control bug exposing health data.
- Benefit: offloads bandwidth for serving images away from our single application server.
- Disadvantage: introduces a third-party dependency and an external account/API key the team must manage and keep out of version control.
- Disadvantage: team must learn a new SDK, which costs setup time now.
- Risk: if the provider's free-tier limits are exceeded or the service has an outage, uploads could fail even though our own server is healthy.
- New work created: adding a storage abstraction layer/helper module so the rest of the app does not depend directly on one provider's SDK, in case the team needs to switch providers later.
- Conditions that may cause the team to revisit this decision: if the client later requires the pharmacy's own server infrastructure for compliance reasons, if the free-tier limits are exceeded once real usage grows, or if the team gains access to reliable persistent server storage that makes local storage viable again.

I. Failure walkthrough

Failure scenario: a customer uploads a valid prescription image. Authentication and file validation succeed, but the request to the cloud storage provider fails (e.g., network timeout or the provider is temporarily unavailable).

1. User action and normal success path: customer selects a valid prescription image and submits the upload; normally the file is sent to the storage provider, a URL/public ID is returned, and the system saves a prescriptions record with status 'Pending Verification'.
2. Component/dependency that fails: the outbound call from the Flask upload route to the cloud storage provider's API fails (timeout or provider error).
3. Failure detection: the storage SDK call raises an exception or returns a non-success response; the Flask route catches this instead of assuming success.
4. What the user sees: a clear, non-technical error message such as 'We could not upload your prescription right now. Please try again in a moment,' with the option to retry — not a raw stack trace.
5. Data at risk: none should persist — because the storage call failed, the route must not create a prescriptions database record pointing to a file that was never actually stored, and must not mark the order as verified or pending on a nonexistent file.

6. What should be logged: timestamp, order ID, customer ID (or hashed identifier), the failed operation name, and the provider's error code/message — but not the prescription image content itself and not full unmasked personal identifiers beyond what is needed to investigate.
7. Immediate mitigation/recovery: the route rolls back any partial database write (no orphaned record), and returns a retry-safe response so the customer can resubmit; the upload is idempotent per attempt so retrying does not create duplicate 'Pending Verification' records for the same order once a later attempt succeeds.
8. Test to verify failure handling: mock/force the storage provider call to fail, then assert that (a) the API response is a clear error, (b) no prescriptions row was inserted, and (c) the order status is unchanged from before the upload attempt.

J. Verification plan

- Walking skeleton: implement a minimal end-to-end path (upload → storage call → database record → pharmacist view) early, before adding edge-case handling, to confirm the chosen storage approach actually works in our deployment environment.
- Test case: automated test that forces the storage call to fail (as in the failure walkthrough) and checks that no orphaned database record is created.
- Test case: automated or manual test confirming a customer cannot retrieve another customer's prescription file/URL.
- Deployment experiment: perform a test redeploy/restart of the app after uploading a test file, and confirm the file is still retrievable afterward (this directly checks the reliability reason behind the decision).
- Security check: confirm that generated file URLs are private/signed and expire, not permanently public, by attempting to access a URL after it should have expired.
- Client/team review: walk the pharmacist-facing verification screen through with the team to confirm the review workflow matches what the client expects.

K. AI disclosure

Tool used: Claude (Anthropic)

Purpose: help structure the ADR into the required sections and draft initial wording for the context, alternatives, comparison matrix, rationale, consequences, and failure walkthrough, based on the prescription-upload feature and storage constraint I identified myself from the team project.

Prompt summary: described the pharmacy project's prescription upload feature and asked for help drafting an ADR comparing local file storage versus managed cloud storage for that feature, following the DCIT 208 IA4 template.

Output used/rejected: [Insert what you kept as-is, and what you rewrote or corrected — e.g., adjusted the assumptions to match your team's actual hosting choice, corrected the GitHub issue link, changed any comparison-matrix reasoning that didn't match how your team actually built the feature].

Verification method: Confirmed with a teammate what hosting platform and storage approach the team is actually using, tested a redeploy to see whether local files were actually lost, checked the free-tier limits on the provider's pricing page.

J. Sketch evidence

Flow Diagram.

Name: Aidoo Robert Kwabena

Student ID: 22337992

Team Name: Techcare Team

Project Name: Pulse Care

Feature: Uploading a photo of a prescription receipt for verification.

Date & Time: 25th June 2026 9:15 PM.

Failure Point: In a case where the an .exe file is uploaded disguised as a prescription Image.

Security Concern: Prescription image contain sensitive medical data and therefore must use HTTPS instead of HTTP (Hyper-Text Transfer Protocol Secure)

Uploading a photo of a prescription receipt for verification.

Customer - Smartphone / Laptop

↓ Types Uniform Resource Locator in browser search bar.

Browser - `www.pharmacyname.com/upload-prescription`

↓ Uniform Resource Locator

DNS Server - Converts Uniform Resource locator to IP address

↓ now the server is located.

HTTPS POST REQUEST - `/upload-prescription`

Details of the Data Sent :

- Customer Name for records
- Phone number for keeping records
- Prescription Image File (format JPG/PNG)

↓ Request arrives at server.

Web Server - Receives POST Request

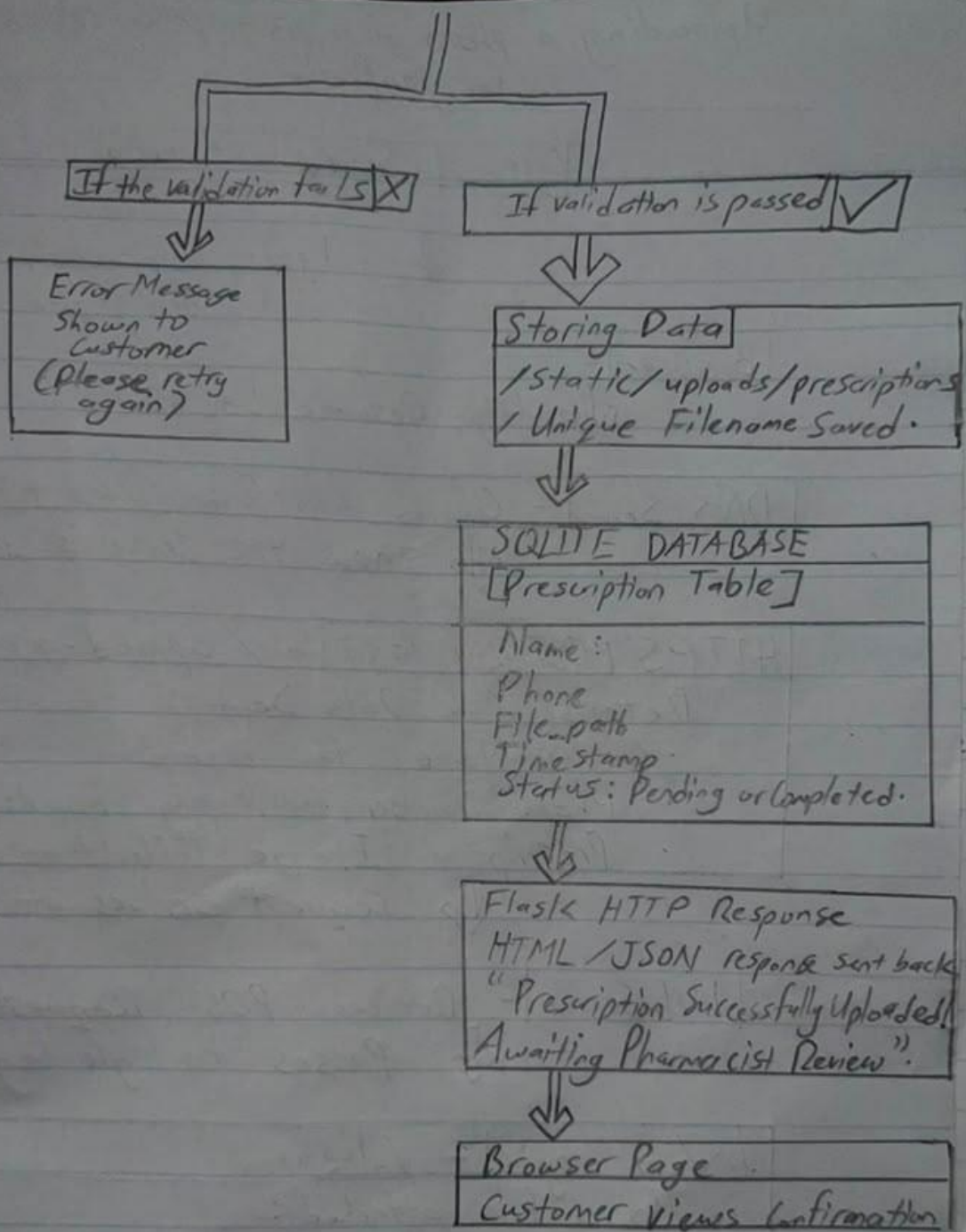
↓ Passes to logic layer

Web Logic - Validation

Is a file attached?

Is it a valid file. α JPG or PNG not .exe?

Is a name and phone number filled in?



Declaration

I confirm that this submission is based on my team's actual client project. I have identified all AI assistance used. I can explain the decision, alternatives, trade-offs, failure scenario and evidence in my own words.

Signature / typed name: AIDOO ROBERT KWABENA